

Додаток 1. Лістинг коду програми

apps/lab1.cpp

```
1 #include <FileSource.hpp>
2 #include <Log.hpp>
3 #include <SymbolStore.hpp>
4 #include <Table.hpp>
5 #include <Tokenizer.hpp>
6 #include <iostream>
7
8 std::string getTypeLabel(SymbolType type) {
9     switch (type) {
10         case SymbolType::Delimiter:
11             return "Delimiter";
12         case SymbolType::Keyword:
13             return "Keyword";
14         case SymbolType::Literal:
15             return "Literal";
16         case SymbolType::Identifier:
17             return "Identifier";
18     }
19 }
20
21 int main(int argc, char* argv[]) {
22     if (argc < 2) {
23         std::cerr << "Usage: lab1 <filename>" << std::endl;
24         return 1;
25     }
26
27     FileSource source(argv[1]);
28     SymbolStore symbols;
29     CharacterAttributes attributes;
30     Tokenizer tokenizer(source, symbols, attributes);
31     tokenizer.scan();
32
33     Table("\nTokens", tokenizer.tokens())
34         .addColumn("Code", "{:>5}", [](const auto& t) { return std::to_string(t.code); })
35         .addColumn("Row", "{:>3}", [](const auto& t) { return std::to_string(t.row + 1); })
36         .addColumn("Col", "{:>3}", [](const auto& t) { return std::to_string(t.column + 1); })
37         .addColumn("Type", "{:<10}", [&](const auto& t) { return getTypeLabel(symbols.lookupType(t.code)); })
38         .addColumn("Value", "{:<30}", [&](const auto& t) { return std::string(symbols.lookup(t.code)); })
39         .print();
40
41     Log("Tokenizer", tokenizer.errors())
42         .setFormatter([](const auto& err) {
43             return std::format("Error [{}:{}]: {}", err.row + 1, err.column + 1, err.message);
44         })
45         .print();
46
47     Table("\nIdentifiers", symbols.identifiers())
48         .addColumn("Code", "{:>5}", [](const auto& pair) { return std::to_string(pair.second); })
49         .addColumn("Value", "{:<30}", [](const auto& pair) { return pair.first; })
50         .print();
51
52     Table("\nLiterals", symbols.literals())
53         .addColumn("Code", "{:>5}", [](const auto& pair) { return std::to_string(pair.second); })
54         .addColumn("Value", "{:<30}", [](const auto& pair) { return pair.first; })
55         .print();
56
57     return 0;
58 }
```

src/CharacterAttributes/CharacterAttributes.hpp

```
1 #pragma once
2
3 #include <array>
4
5 #include "Constants.hpp"
6
7 enum class Attribute {
8     Whitespace,
9     Digit,
10    Letter,
11    Delimiter,
12    Comment,
13    Invalid,
14 };
15
16 class CharacterAttributes {
17 private:
18     std::array<Attribute, ASCII_TABLE_SIZE> attributes;
19
20 public:
21     CharacterAttributes();
22
23     [[nodiscard]] Attribute lookup(char character) const;
24 };
```

src/CharacterAttributes/CharacterAttributes.cpp

```
1 #include "CharacterAttributes.hpp"
2
3 CharacterAttributes::CharacterAttributes() {
4     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
5         attributes[i] = Attribute::Invalid;
6     }
7
8     // Whitespace
9     attributes[' '] = Attribute::Whitespace;
10    attributes['\t'] = Attribute::Whitespace;
11    attributes['\n'] = Attribute::Whitespace;
12    attributes['\v'] = Attribute::Whitespace;
13    attributes['\f'] = Attribute::Whitespace;
14    attributes['\r'] = Attribute::Whitespace;
15
16    // Digits
17    for (char ch = '0'; ch <= '9'; ch++) {
18        attributes[ch] = Attribute::Digit;
19    }
20
21    // Letters (A-Z)
22    for (char ch = 'A'; ch <= 'Z'; ch++) {
23        attributes[ch] = Attribute::Letter;
24    }
25
26    // Delimiters
27    attributes[';'] = Attribute::Delimiter;
28    attributes['.'] = Attribute::Delimiter;
29    attributes['='] = Attribute::Delimiter;
30    attributes['\''] = Attribute::Delimiter;
31    attributes[','] = Attribute::Delimiter;
32    attributes[')'] = Attribute::Delimiter;
33    attributes['$'] = Attribute::Delimiter;
34
35    // Comment start
36    attributes['('] = Attribute::Comment;
37 }
38
39 Attribute CharacterAttributes::lookup(char character) const {
40     return attributes[static_cast<unsigned char>(character)];
41 }
```

src/Constants/Constants.hpp

```
1 #pragma once
2
3 #include <cstdint>
4
5 constexpr size_t ASCII_TABLE_SIZE = 256;
```

src/Log/Log.hpp

```
1 #pragma once
2
3 #include <functional>
4 #include <iostream>
5 #include <string>
6 #include <vector>
7
8 template <typename T>
9 class Log {
10 private:
11     std::string name;
12     std::vector<T> data;
13     std::function<std::string(const T&)> formatter;
14     std::ostream& out;
15
16 public:
17     Log(std::string name, std::vector<T> data, std::ostream& out = std::cout);
18
19     Log& setFormatter(std::function<std::string(const T&)> formatter);
20
21     void print() const;
22 };
23
24 #include "Log.tpp"
```

src/Log/Log.hpp

```
1 template <typename T>
2 Log<T>::Log(std::string name, std::vector<T> data, std::ostream& out) :
3     name(std::move(name)), data(std::move(data)), out(out) {}
4
5 template <typename T>
6 Log<T>& Log<T>::setFormatter(std::function<std::string(const T&)> formatter) {
7     this->formatter = std::move(formatter);
8
9     return *this;
10 }
11
12 template <typename T>
13 void Log<T>::print() const {
14     for (const auto& item : data) {
15         out << name << " | " << formatter(item) << "\n";
16     }
17 }
```

src/Source/Source.hpp

```
1 #pragma once
2
3 #include <SourcePosition.hpp>
4
5 class Source {
6 protected:
7     SourcePosition position;
8
9 public:
10     Source();
11     Source(SourcePosition position);
12
13     virtual ~Source() = default;
14
15     virtual char current() = 0;
16     virtual char read() = 0;
17     virtual bool done() = 0;
18
19     [[nodiscard]] unsigned int row() const;
20     [[nodiscard]] unsigned int column() const;
21 };
```

src/Source/Source.cpp

```
1 #include "Source.hpp"
2
3 Source::Source() = default;
4
5 Source::Source(SourcePosition position) : position(position) {}
6
7 unsigned int Source::row() const {
8     return position.row();
9 }
10
11 unsigned int Source::column() const {
12     return position.column();
13 }
```


src/Source/FileSource/FileSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include <fstream>
5 #include "Source.hpp"
6
7 class FileSource : public Source {
8 private:
9     std::ifstream file;
10     char character;
11
12 public:
13     FileSource(const std::string& filePath);
14
15     char current() override;
16     char read() override;
17     bool done() override;
18 };
```

src/Source/FileSource/FileSource.cpp

```
1 #include "FileSource.hpp"
2
3 FileSource::FileSource(const std::string& filePath) :
4     file(filePath),
5     character(static_cast<char>(file.get())) {}
6
7 char FileSource::current() {
8     return character;
9 }
10
11 char FileSource::read() {
12     position.advance(character);
13     character = static_cast<char>(file.get());
14
15     return character;
16 }
17
18 bool FileSource::done() {
19     return file.eof();
20 }
```

src/Source/StringSource/StringSource.hpp

```
1 #pragma once
2
3 #include <string>
4 #include "Source.hpp"
5
6 class StringSource : public Source {
7 private:
8     std::string source;
9     std::string::iterator iterator;
10
11 public:
12     StringSource(std::string source);
13
14     char current() override;
15     char read() override;
16     bool done() override;
17 };
```

src/Source/StringSource/StringSource.cpp

```
1 #include "StringSource.hpp"
2
3 StringSource::StringSource(std::string source) :
4     source(std::move(source)),
5     iterator(this->source.begin()) {}
6
7 char StringSource::current() {
8     return *iterator;
9 }
10
11 char StringSource::read() {
12     position.advance(*iterator);
13     iterator++;
14
15     return *iterator;
16 }
17
18 bool StringSource::done() {
19     return iterator == source.end();
20 }
```

src/SourcePosition/SourcePosition.hpp

```
1 #pragma once
2
3 #include <cstdint>
4
5 class SourcePosition {
6 private:
7     uint8_t tabSize = 4;
8
9     unsigned int _row = 0;
10    unsigned int _column = 0;
11
12 public:
13     SourcePosition();
14     SourcePosition(uint8_t tabSize);
15
16     void advance(char character);
17     [[nodiscard]] unsigned int row() const;
18     [[nodiscard]] unsigned int column() const;
19 };
```

src/SourcePosition/SourcePosition.cpp

```
1 #include "SourcePosition.hpp"
2
3 SourcePosition::SourcePosition() : SourcePosition(4) {}
4
5 SourcePosition::SourcePosition(uint8_t tabSize) : tabSize(tabSize) {};
6
7 void SourcePosition::advance(char character) {
8     switch (character) {
9         case '\n':
10             _row++;
11             _column = 0;
12             break;
13
14         case '\t':
15             _column += tabSize;
16             break;
17
18         default:
19             _column++;
20             break;
21     }
22 }
23
24 unsigned int SourcePosition::row() const {
25     return _row;
26 }
27
28 unsigned int SourcePosition::column() const {
29     return _column;
30 }
```

src/SymbolStore/SymbolStore.hpp

```
1  #pragma once
2
3  #include <string>
4  #include <unordered_map>
5  #include <vector>
6
7  #include "Constants.hpp"
8
9  enum class SymbolType {
10     Delimiter,
11     Keyword,
12     Literal,
13     Identifier,
14 };
15
16 const size_t DELIMITERS_OFFSET = 0;
17 const size_t KEYWORDS_OFFSET = ASCII_TABLE_SIZE;
18 const size_t LITERALS_OFFSET = 500;
19 const size_t IDENTIFIERS_OFFSET = 1000;
20 const size_t MAX_TOKENS = 2000;
21
22 class SymbolStore {
23 private:
24     std::array<std::string, MAX_TOKENS> symbols;
25     std::unordered_map<std::string, size_t> _keywords;
26     std::unordered_map<std::string, size_t> _identifiers;
27     std::unordered_map<std::string, size_t> _literals;
28
29     void declareDelimiter(char character);
30     void declareKeyword(const std::string& keyword);
31
32 public:
33     SymbolStore();
34
35     size_t resolveKeyword(const std::string& keyword);
36     size_t resolveIdentifier(const std::string& identifier);
37     size_t resolveLiteral(const std::string& literal);
38
39     [[nodiscard]] bool isKeyword(const std::string& token) const;
40     [[nodiscard]] const std::string& lookup(size_t code) const;
41     [[nodiscard]] SymbolType lookupType(size_t code) const;
42     [[nodiscard]] std::vector<std::pair<std::string, size_t>> identifiers() const;
43     [[nodiscard]] std::vector<std::pair<std::string, size_t>> literals() const;
44 };
```

src/SymbolStore/SymbolStore.cpp

```
1 #include "SymbolStore.hpp"
2
3 #include <algorithm>
4 #include <format>
5 #include <stdexcept>
6
7 SymbolStore::SymbolStore() {
8     // Initialize ASCII symbols
9     for (size_t i = 0; i < ASCII_TABLE_SIZE; i++) {
10         declareDelimiter(static_cast<char>(i));
11     }
12
13     // Initialize keywords
14     declareKeyword("PROGRAM");
15     declareKeyword("CONST");
16     declareKeyword("BEGIN");
17     declareKeyword("END");
18     declareKeyword("EXP");
19 }
20
21 void SymbolStore::declareDelimiter(const char character) {
22     symbols[static_cast<unsigned char>(character)] = std::string(1, character);
23 }
24
25 void SymbolStore::declareKeyword(const std::string& keyword) {
26     if (_keywords.contains(keyword)) {
27         throw std::invalid_argument(std::format("Keyword '{}{}' is already declared", keyword));
28     }
29
30     size_t code = KEYWORDS_OFFSET + _keywords.size();
31
32     if (code >= LITERALS_OFFSET) {
33         throw std::overflow_error("Exceeded maximum number of keywords");
34     }
35
36     _keywords[keyword] = code;
37     symbols[code] = keyword;
38 }
39
40 size_t SymbolStore::resolveKeyword(const std::string& keyword) {
41     if (!_keywords.contains(keyword)) {
42         throw std::invalid_argument(std::format("'{}{}' is not a keyword", keyword));
43     }
44
45     return _keywords[keyword];
46 }
47
48 size_t SymbolStore::resolveIdentifier(const std::string& identifier) {
49     if (!_identifiers.contains(identifier)) {
50         size_t code = IDENTIFIERS_OFFSET + _identifiers.size();
51
52         if (code >= MAX_TOKENS) {
53             throw std::overflow_error("Exceeded maximum number of identifiers");
54         }
55
56         _identifiers[identifier] = code;
57         symbols[code] = identifier;
58     }
59
60     return _identifiers[identifier];
61 }
62
```



```

63 size_t SymbolStore::resolveLiteral(const std::string& literal) {
64     if (!_literals.contains(literal)) {
65         size_t code = LITERALS_OFFSET + _literals.size();
66
67         if (code >= IDENTIFIERS_OFFSET) {
68             throw std::overflow_error("Exceeded maximum number of literals");
69         }
70
71         _literals[literal] = code;
72         symbols[code] = literal;
73     }
74
75     return _literals[literal];
76 }
77
78 bool SymbolStore::isKeyword(const std::string& token) const {
79     return _keywords.contains(token);
80 }
81
82 SymbolType SymbolStore::lookupType(size_t code) const {
83     if (code < KEYWORDS_OFFSET) {
84         return SymbolType::Delimiter;
85     }
86
87     if (code < LITERALS_OFFSET) {
88         return SymbolType::Keyword;
89     }
90
91     if (code < IDENTIFIERS_OFFSET) {
92         return SymbolType::Literal;
93     }
94
95     if (code < MAX_TOKENS) {
96         return SymbolType::Identifier;
97     }
98
99     throw std::out_of_range(std::format("Symbol code {} is out of range", code));
100 }
101
102 const std::string& SymbolStore::lookup(size_t code) const {
103     if (code >= symbols.size()) {
104         throw std::out_of_range(std::format("Symbol code {} is out of range", code));
105     }
106
107     return symbols[code];
108 }
109
110 std::vector<std::pair<std::string, size_t>> SymbolStore::identifiers() const {
111     std::vector<std::pair<std::string, size_t>> result;
112
113     for (const auto& pair : _identifiers) {
114         result.emplace_back(pair.first, pair.second);
115     }
116
117     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) { return a.second <
118
119     return result;
120 }
121
122 std::vector<std::pair<std::string, size_t>> SymbolStore::literals() const {
123     std::vector<std::pair<std::string, size_t>> result;
124
125     for (const auto& pair : _literals) {
126         result.emplace_back(pair.first, pair.second);
127     }
128
129     std::sort(result.begin(), result.end(), [](const auto& a, const auto& b) { return a.second <

```

```
130
131     return result;
132 }
```

src/Table/Table.hpp

```
1  #pragma once
2
3  #include <format>
4  #include <functional>
5  #include <iostream>
6  #include <string>
7  #include <vector>
8
9  template <typename T>
10 class Table {
11 private:
12     struct Column {
13         std::string name;
14         std::string format;
15         std::function<std::string(const T&)> extractor;
16     };
17
18     std::string name;
19     std::vector<T> data;
20     std::vector<Column> columns;
21     std::ostream& out;
22
23     [[nodiscard]] std::string border() const;
24     [[nodiscard]] std::string header() const;
25     [[nodiscard]] std::string row(const T& item) const;
26     [[nodiscard]] size_t columnWidth(const Column& col) const;
27     static size_t parseWidth(const std::string& fmt);
28
29 public:
30     Table(std::string name, std::vector<T> data, std::ostream& out = std::cout);
31
32     Table& addColumn(
33         const std::string& colName,
34         const std::string& format,
35         std::function<std::string(const T&)> extractor
36     );
37
38     void print() const;
39 };
40
41 #include "Table.hpp"
```

```
1  template <typename T>
2  Table<T>::Table(std::string name, std::vector<T> data, std::ostream& out) :
3      name(std::move(name)), data(std::move(data)), out(out) {}
4
5  template <typename T>
6  Table<T>& Table<T>::addColumn(
7      const std::string& colName,
8      const std::string& format,
9      std::function<std::string(const T&)> extractor
10 ) {
11     columns.push_back({colName, format, std::move(extractor)});
12
13     return *this;
14 }
15
16 template <typename T>
17 void Table<T>::print() const {
18     if (!name.empty()) {
19         out << name << "\n";
20     }
21
22     std::string borderLine = border();
23
24     out << borderLine << "\n";
25     out << header() << "\n";
26     out << borderLine << "\n";
27
28     for (const auto& item : data) {
29         out << row(item) << "\n";
30     }
31
32     out << borderLine << "\n";
33 }
34
35 template <typename T>
36 std::string Table<T>::border() const {
37     std::string result = "+";
38
39     for (const auto& col : columns) {
40         size_t width = columnWidth(col);
41         result += std::string(width + 2, '-') + "+";
42     }
43
44     return result;
45 }
46
47 template <typename T>
48 std::string Table<T>::header() const {
49     std::string result = "|";
50
51     for (const auto& column : columns) {
52         result += std::format(" {:<{}} |", column.name, columnWidth(column));
53     }
54
55     return result;
56 }
57
58 template <typename T>
59 std::string Table<T>::row(const T& item) const {
60     std::string result = "|";
61
62     for (const auto& column : columns) {
```

```

63         std::string value = column.extractor(item);
64         result += " " + std::vformat(column.format, std::make_format_args(value)) + " |";
65     }
66
67     return result;
68 }
69
70 template <typename T>
71 size_t Table<T>::columnWidth(const Column& column) const {
72     return std::max(parseWidth(column.format), column.name.length());
73 }
74
75 template <typename T>
76 size_t Table<T>::parseWidth(const std::string& fmt) {
77     size_t width = 0;
78
79     for (char chr : fmt) {
80         if (chr >= '0' && chr <= '9') {
81             width = (width * 10) + (chr - '0');
82         }
83     }
84
85     return width;
86 }

```

src/Tokenizer/Tokenizer.hpp

```
1  #pragma once
2
3  #include <unordered_map>
4  #include <vector>
5
6  #include "CharacterAttributes.hpp"
7  #include "Source.hpp"
8  #include "SymbolStore.hpp"
9
10 struct Token {
11     size_t code;
12     size_t row;
13     size_t column;
14 };
15
16 struct Error {
17     std::string message;
18     size_t row;
19     size_t column;
20 };
21
22 class Tokenizer {
23 private:
24     Source& source;
25     SymbolStore& symbols;
26     CharacterAttributes& attributes;
27
28     char character;
29     std::string token;
30     size_t code;
31
32     std::vector<Token> _tokens;
33     std::vector<Error> _errors;
34
35     void scanWhitespaces();
36     void scanInteger();
37     void scanString();
38     void scanComment();
39     void scanDelimiter();
40     void scanInvalid();
41
42 public:
43     Tokenizer(Source& source, SymbolStore& symbols, CharacterAttributes& attributes);
44
45     void scan();
46
47     void addToken(const Token& token);
48     void addError(const Error& error);
49
50     [[nodiscard]] const std::vector<Token>& tokens() const;
51     [[nodiscard]] const std::vector<Error>& errors() const;
52 };
```

src/Tokenizer/Tokenizer.cpp

```
1  #include "Tokenizer.hpp"
2
3  #include <format>
4
5  #include "CharacterAttributes.hpp"
6
7  // Automata state: START
8  Tokenizer::Tokenizer(Source& source, SymbolStore& symbols, CharacterAttributes& attributes) :
9      source(source), symbols(symbols), attributes(attributes), character(source.current()) {}
10
11 void Tokenizer::scan() {
12     // Automata state: LOOP
13     while (!source.done()) {
14         switch (attributes.lookup(character)) {
15             // Automata state: WHITESPACE
16             case Attribute::Whitespace:
17                 scanWhitespaces();
18                 break;
19
20             // Automata state: INTEGER
21             case Attribute::Digit:
22                 scanInteger();
23                 break;
24
25             // Automata state: STRING
26             case Attribute::Letter:
27                 scanString();
28                 break;
29
30             // Automata state: COMMENT_START
31             case Attribute::Comment:
32                 scanComment();
33                 break;
34
35             // Automata state: DELIMITER
36             case Attribute::Delimiter:
37                 scanDelimiter();
38                 break;
39
40             // Automata state: ERROR
41             case Attribute::Invalid:
42                 scanInvalid();
43                 break;
44         }
45     }
46     // Automata state: END
47 }
48
49 void Tokenizer::scanWhitespaces() {
50     while (!source.done() && attributes.lookup(character) == Attribute::Whitespace) {
51         character = source.read();
52     }
53 }
54
55 void Tokenizer::scanInteger() {
56     while (!source.done() && attributes.lookup(character) == Attribute::Digit) {
57         token += character;
58         character = source.read();
59     };
60
61     // Automata state: WRITE
62     code = symbols.resolveLiteral(token);
```

```

63
64     addToken({
65         .code = code,
66         .row = source.row(),
67         .column = source.column() - token.length(), // Point to first character of the token
68     });
69
70     token.clear();
71 }
72
73 void Tokenizer::scanString() {
74     while (!source.done() &&
75         (attributes.lookup(character) == Attribute::Letter || attributes.lookup(character) ==
76         token += character;
77         character = source.read();
78     }
79
80     // Automata state: WRITE
81     if (symbols.isKeyword(token)) {
82         code = symbols.resolveKeyword(token);
83     } else {
84         code = symbols.resolveIdentifier(token);
85     }
86
87     addToken({
88         .code = code,
89         .row = source.row(),
90         .column = source.column() - token.length(), // Point to first character of the token
91     });
92
93     token.clear();
94 }
95
96 void Tokenizer::scanComment() {
97     size_t startRow = source.row();
98     size_t startCol = source.column();
99
100     character = source.read();
101
102     // Automata state: WRITE
103     if (character != '*') {
104         addToken({
105             .code = static_cast<size_t>('('),
106             .row = startRow,
107             .column = startCol,
108         });
109
110         return;
111     }
112
113     character = source.read();
114
115     // Automata state: COMMENT
116     while (!source.done()) {
117         // Automata state: COMMENT_CLOSE
118         if (character == '*') {
119             character = source.read();
120
121             // Automata state: COMMENT_END
122             if (character == ')') {
123                 character = source.read();
124
125                 return;
126             }
127         } else {
128             character = source.read();
129         }

```



```

130     }
131
132     // Automata state: ERROR
133     addError({
134         .message = "Comment not closed",
135         .row = startRow,
136         .column = startCol,
137     });
138 }
139
140 void Tokenizer::scanDelimiter() {
141     addToken({
142         .code = static_cast<size_t>(character),
143         .row = source.row(),
144         .column = source.column(),
145     });
146
147     character = source.read();
148 }
149
150 void Tokenizer::scanInvalid() {
151     addError({
152         .message = std::format("Invalid character '{}'", character),
153         .row = source.row(),
154         .column = source.column(),
155     });
156
157     character = source.read();
158 }
159
160 const std::vector<Token>& Tokenizer::tokens() const {
161     return _tokens;
162 }
163
164 const std::vector<Error>& Tokenizer::errors() const {
165     return _errors;
166 }
167
168 void Tokenizer::addToken(const Token& token) {
169     _tokens.push_back(token);
170 }
171
172 void Tokenizer::addError(const Error& error) {
173     _errors.push_back(error);
174 }

```

src/TokensView/TokensView.hpp

```
1 #pragma once
2
3 #include <iostream>
4 #include <vector>
5
6 #include "SymbolStore.hpp"
7 #include "Tokenizer.hpp"
8
9 class TokensView {
10 private:
11     std::ostream& out;
12
13     static std::string typeLabel(SymbolType type);
14
15 public:
16     TokensView(std::ostream& out);
17
18     void print(const std::vector<Token>& tokens, const SymbolStore& symbols) const;
19 };
```

src/TokensView/TokensView.cpp

```
1  #include "TokensView.hpp"
2
3  #include <format>
4
5  TokensView::TokensView(std::ostream& out) : out(out) {}
6
7  std::string TokensView::typeLabel(SymbolType type) {
8      switch (type) {
9          case SymbolType::Delimiter:
10             return "Delimiter";
11          case SymbolType::Keyword:
12             return "Keyword";
13          case SymbolType::Literal:
14             return "Literal";
15          case SymbolType::Identifier:
16             return "Identifier";
17      }
18  }
19
20  void TokensView::print(const std::vector<Token>& tokens, const SymbolStore& symbols) const {
21      std::string border = "+-----+-----+-----+-----+-----+";
22
23      std::string header =
24          std::format("| {:>5} | {:>3} | {:>3} | {:<10} | {:<7} |", "Code", "Row", "Col", "Type", "Value");
25
26      out << border << "\n";
27      out << header << "\n";
28      out << border << "\n";
29
30      for (const auto& token : tokens) {
31          auto value = symbols.lookup(token.code);
32          auto type = typeLabel(symbols.lookupType(token.code));
33
34          out << std::format(
35              "| {:>5} | {:>3} | {:>3} | {:<10} | {:<7} |", token.code, token.row + 1, token.col + 1,
36              value
37          )
38              << "\n";
39      }
40
41      out << border << "\n";
42  }
```